Guía Maestra de Estudio — 20 katas para Claude Certified Architect

Workshop ia-claude · concept-first cert prep

raul.alzate@sofka.com.co

Índice

- Cómo usar esta guía
- Mapa temático
- Cómo leer un kata en 10 minutos
- Kata 01 — Bucle Agéntico Determinista
  - Concepto
  - Por qué importa
  - Modelo mental
  - Ejemplo mínimo
  - Anti-patrón
  - Argumento de certificación
  - Auto-evaluación
- Kata 02 — Guardarraíles Deterministas con `PreToolUse`
  - Concepto
  - Por qué importa
  - Modelo mental
  - Ejemplo mínimo
  - Anti-patrón
  - Argumento de certificación
  - Auto-evaluación
- Kata 03 — Normalización con `PostToolUse`
  - Concepto
  - Por qué importa
  - Modelo mental
  - Ejemplo mínimo
  - Anti-patrón
  - Argumento de certificación
  - Auto-evaluación
- Kata 04 — Aislamiento Estricto de Subagentes (Hub-and-Spoke)
  - Concepto
  - Por qué importa
  - Modelo mental
  - Ejemplo mínimo
  - Anti-patrón
  - Argumento de certificación
  - Auto-evaluación
- Kata 05 — Extracción Estructurada Defensiva con JSON Schema
  - Concepto
  - Por qué importa
  - Modelo mental
  - Ejemplo mínimo
  - Anti-patrón
  - Argumento de certificación
  - Auto-evaluación
- Kata 06 — Errores Estructurados en MCP
  - Concepto
  - Por qué importa
  - Modelo mental
  - Ejemplo mínimo
  - Anti-patrón

Argumento de certificación
Auto-evaluación
Kata 07 — Exploración Segura con Plan Mode
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 08 — Memoria Jerárquica con `CLAUDE.md`
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 09 — Reglas Condicionales por Ruta
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 10 — Optimización Económica con Prefix Caching
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 11 — Mitigación de Dilución Softmax (Edge Placement + Compactación)
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 12 — Prompt Chaining Multi-Pass
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación

Kata 13 — Code Review Headless en CI/CD
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 14 — Few-Shot para Calibrar Bordes
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 15 — Evaluación Crítica y Auto-Corrección
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 16 — Protocolo de Handoff a Humano
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 17 — Procesamiento Masivo con Message Batches API
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 18 — Scratchpad Persistente
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 19 — Investigación Adaptativa (Descomposición Dinámica)
- Concepto

Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 20 — Preservación de Provenance
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Glosario
Cheat sheet (una página)
Antes del examen, repite

Cómo usar esta guía

Este documento condensa los 20 katas del workshop `ia-claude` en una sola guía imprimible para preparar el examen oral del **Claude Certified Architect**. Cada kata aparece con la misma estructura:

1. **Concepto** — la idea en una página.
2. **Por qué importa** — la falla que evita.
3. **Modelo mental** — cómo pensar el patrón.
4. **Ejemplo mínimo** — pseudocódigo o snippet conceptual.
5. **Anti-patrón** — el error común y por qué falla.
6. **Argumento de certificación** — los puntos a defender en voz alta.
7. **Auto-evaluación** — preguntas con respuestas propuestas.

El **entregable real** de cada kata vive en `katas/kata_NNN_<slug>/notebook.ipynb` y demuestra el patrón contra la API de Anthropic. Esta guía es el complemento conceptual que se lee imprimido, sin pantallas.

Mapa temático

Los 20 katas se organizan en hilos:

Hilo	Katas
Determinismo	01, 02, 06, 13
Schemas y contratos	05, 13, 15, 16, 20
Economía de contexto	03, 08, 09, 10, 11, 12
Memoria	08, 11, 18, 19
Aislamiento	04, 12, 19
Human-in-the-loop	07, 15, 16

Una decisión técnica suele tocar varios hilos a la vez. Las conexiones inter-katas se enuncian al final de cada sección “Argumento de certificación”.

Cómo leer un kata en 10 minutos

1. Lee la sección **Concepto** y formula con tus palabras la regla en una sola frase.
2. Cubre la sección **Anti-patrón** con la mano y trata de anticiparla.
3. Lee la **Auto-evaluación** sin mirar la respuesta. Compara después.
4. Si las respuestas no encajan, vuelve al **Modelo mental** — ahí está la falla de comprensión.
5. Salta al notebook si quieres ver el código corriendo.

Kata 01 — Bucle Agéntico Determinista

Concepto

Un agente Claude no termina cuando "dice" que terminó: termina cuando la API devuelve un `stop_reason` terminal. El bucle agéntico es un `while` controlado por metadatos estructurados (`stop_reason`), no por el contenido del texto que genera el modelo.

Tres ramas posibles por turno:

- `tool_use` → ejecutar la herramienta, anexar el resultado al historial, iterar.
- `end_turn` → detener el bucle, devolver respuesta final.
- cualquier otro (`max_tokens`, ausente, desconocido) → detener con motivo explícito; nunca seguir adivinando.

Por qué importa

El primer error de un sistema agéntico es decidir terminación con `if "done" in text`. El día que el modelo escribe "we are done" en mitad de una cadena de herramientas, el agente corta antes de tiempo y el negocio paga la diferencia. La certificación exige que el control de flujo sea **observable y reproducible**; sólo `stop_reason` cumple eso.

Modelo mental

- El texto del modelo es **carga útil**, no señal de control.
- El bucle es un autómata: estado = historial; transición = `stop_reason`.
- Cada iteración deja una entrada en un log estructurado (índice, señal, rama, herramienta, motivo de fin) suficiente para reproducir la traza sin re-ejecutar.

Ejemplo mínimo

```
while True:
    resp = client.messages.create(messages=history, tools=tools, ...)
    if resp.stop_reason == "tool_use":
        result = run_tool(resp.content)            # determinista
        history.append(result)
        continue
    if resp.stop_reason == "end_turn":
        return resp                                # éxito
    raise UnhandledStop(resp.stop_reason)          # nunca silencioso
```

Ningún `re.search`, ningún `"done" in text`. Sólo metadatos.

Anti-patrón

Decidir terminación, despacho de herramientas o reintentos a partir del texto generado (regex, `in`, `contains`). Falla silenciosamente cuando el modelo parafrasea, idioma cambia, o el usuario inyecta la frase trampa. Es la forma más común de "agente que casi funciona en demo y se rompe en producción".

Argumento de certificación

- Sé explicar por qué `stop_reason` es la única fuente legítima de control.
- Sé enumerar los `stop_reason` posibles y la respuesta correcta para cada uno.
- Sé describir el log mínimo que hace una corrida auditable y reproducible.

Auto-evaluación

1. ¿Qué ocurre si el modelo devuelve `stop_reason=max_tokens`? ¿Continúo, abortó, reintento?

Aborto con motivo explícito (`unhandled:max_tokens`), como muestra la celda §3.4. No reintento automático: la respuesta está truncada, así que reintentar sin cambiar nada repetiría el truncamiento. Lo correcto es subir el cap de `max_tokens` para esa llamada, o pedir al modelo que continúe en un turno nuevo (turno explícito iniciado por el cliente, no por el bucle). El bucle deja la decisión al llamador y registra la causa en el log.

2. Si una herramienta lanza excepción, ¿cómo lo veo en el historial sin romper el invariante "control por señal"?

Capturo la excepción dentro del bloque `tool_use`, la convierto en un `tool_result` con campo `error`, y la anexo al historial igual que cualquier otro resultado (en `run_loop_signal` lo hago en el bloque `try/except`). El bucle continúa: la próxima iteración mostrará al modelo el error tipado y le permitirá replanificar. El control sigue siendo `stop_reason`; el fallo es payload, no señal.

3. ¿Qué información mínima debe registrar el log para reconstruir la traza completa sin volver a llamar al modelo?

Por iteración: índice, `stop_reason`, rama tomada (`dispatch` / `halt`), nombre de la herramienta cuando aplique, status (`ok` / `error`) de la ejecución y, en la iteración terminal, la causa de corte (`end_turn` o `unhandled:<reason>`). Es exactamente lo que produce `Logger.show()`. Para replicar la respuesta del modelo bit-a-bit haría falta también el guion crudo, pero el log de control basta para auditar la lógica del agente.

Kata 02 — Guardarraíles Deterministas con `PreToolUse`

Concepto

Un hook `PreToolUse` se ejecuta **antes** de que la herramienta corra. Puede permitir, denegar o pedir aprobación humana basándose en el payload exacto del `tool_use`. La política se aplica en código, no en el prompt.

Por qué importa

Pedirle al modelo "no borres datos de producción" en el system prompt es una sugerencia, no un control. Un `PreToolUse` que rechaza `DROP TABLE` o `rm -rf /` es un control verificable. La diferencia define si el sistema es auditable o sólo "casi seguro".

Modelo mental

- El prompt **sugiere**; el hook **aplica**.
- Veredictos tipados: `allow | deny | ask_human`, siempre con razón.
- La política vive en archivo (JSON/YAML) recargable en caliente. El modelo no la ve para que no pueda persuadirla.
- Cada veredicto produce evento estructurado: herramienta, args, decisión, regla.

Ejemplo mínimo

```
def pretool_hook(tool_name, tool_input) -> Verdict:
    if tool_name == "shell" and re.search(r"\brm -rf\b", tool_input["cmd"]):
        return Verdict("deny", reason="POL-DELETE-001")
    if tool_name == "refund" and tool_input["amount"] > 1000:
        return Verdict("ask_human", reason="POL-REFUND-LIMIT")
    return Verdict("allow")
```

Si el hook deniega, el modelo recibe un `tool_result` con error tipado y puede replanificar. La acción peligrosa nunca se ejecuta.

Anti-patrón

"Defensa en prompt": párrafos en el system prompt rogando al modelo no hacer X. Funciona el 95 % del tiempo y falla justo cuando importa: jailbreak, prompt injection en datos del usuario, comportamiento emergente de un tool nuevo.

Argumento de certificación

- Sé distinguir control suave (prompt) de control duro (hook).
- Sé describir los tres veredictos y cuándo emitir cada uno.
- Sé justificar por qué la política vive fuera del prompt y se recarga en caliente.

Auto-evaluación

1. Si la política cambia mientras hay sesión activa, ¿cómo aseguro que el hook use la versión nueva sin reiniciar el agente?

`pretool_check` lee `POLICY` cada vez que se invoca. Si reemplazo el dict (o lo recargo desde JSON), la siguiente llamada al hook ya ve la nueva versión. La sesión del modelo no necesita reiniciarse — el hook es el único que conoce la política y vive del lado del cliente.

2. ¿Cómo distingo "denegar y dejar continuar" de "denegar y escalar"?

Son dos veredictos distintos. `deny` produce un `tool_result` con `error_code`; el modelo lo recibe como contexto y

replanifica (puede decirle al usuario que no se puede). `ask_human` produce un `tool_result` con `requires_approval=true` y, en producción, despacha a la cola humana (ver Kata 16).

3. ¿Qué prueba reintroduce el anti-patrón a propósito para verificar que el hook lo bloquea?

La celda §4 con el ataque social (CFO de emergencia). Si el bucle sin gate ejecuta el reembolso, demuestra que el prompt-only falla; si el bucle con gate registra `verdict=deny`, demuestra que la política aplicada externamente sigue siendo robusta.

Kata 03 — Normalización con `PostToolUse`

Concepto

Un hook `PostToolUse` corre **después** de la herramienta y **antes** de que el resultado llegue al contexto del modelo. Limpia, traduce códigos arcanos y proyecta el payload a un JSON mínimo y predecible.

Por qué importa

Si la herramienta devuelve XML legacy de 5 KB con códigos `0xA7`, el modelo gasta tokens para parsearlo cada turno y, peor, alucina cuando el formato cambia. Normalizar una vez, en un sitio determinista, ahorra contexto y elimina ambigüedad.

Modelo mental

- El modelo no debería ver formatos crudos jamás. Sólo JSON canónico.
- Los códigos numéricos/legacy se mapean a strings legibles (`0xA7` → `"timeout"`).
- El hook es puro: misma entrada → mismo JSON. Probable de testear sin LLM.
- Mide en tokens: el run normalizado debe ser estrictamente más barato que el crudo.

Ejemplo mínimo

```
def posttool_hook(tool_name, raw) -> dict:
    if tool_name != "legacy_lookup":
        return raw
    return {
        "id": raw["@id"],
        "status": STATUS_CODE_MAP.get(raw["s"], "unknown"),
        "amount": float(raw["amt"]),
    }
```

El historial almacena el JSON limpio; el XML original se descarta del contexto.

Anti-patrón

Inyectar el payload crudo y “que el modelo se las arregle”. Cada turno re-paga los tokens del parseo y la primera vez que el proveedor cambia el formato (o añade un código) el agente alucina o falla en silencio.

Argumento de certificación

- Sé describir cuándo `PostToolUse` aplica vs `PreToolUse`.
- Sé enunciar la regla “el modelo nunca ve datos crudos heterogéneos”.
- Sé medir el ahorro de tokens y tasa de mala interpretación con/sin hook.

Auto-evaluación

1. ¿Qué pasa si el hook recibe un código que no está en el mapa?

Devuelvo `"unknown"` para no romper el contrato (el campo siempre existe). En producción, además, registro un evento de “código no mapeado” para que el equipo lo revise. Nunca dejo que el modelo “adivine” qué significa.

2. ¿Cómo pruebo el hook sin levantar el modelo?

`posttool_normalize` es una función pura sobre dicts. Le paso un fixture y compruebo el resultado con `assert`. Cero llamadas a la API. Si más adelante el formato legacy cambia, agrego entradas al fixture y al `STATUS_MAP`.

3. ¿Qué métrica concreta demuestra que el hook redujo “carga cognitiva”?

`response.usage.input_tokens` en la segunda llamada (la que ve el `tool_result`). Comparé el run normalizado vs el crudo en §4. Para evidencia más fuerte, podría correr un eval con N invocaciones y medir tasa de respuestas correctas.

Kata 04 — Aislamiento Estricto de Subagentes (Hub-and-Spoke)

Concepto

Un coordinador descompone una tarea grande y lanza subagentes. Cada subagente recibe **sólo** el prompt mínimo que necesita y devuelve **sólo** un JSON tipado. Cero memoria compartida, cero historial heredado.

Por qué importa

Heredar el historial completo del coordinador parece "ayudar al subagente a entender el contexto". En la práctica diluye su atención, filtra políticas sensibles, y hace los fallos imposibles de bisecar (¿de qué turno ancestral viene este sesgo?).

Modelo mental

- Coordinador = hub. Subagentes = spokes. No hay aristas spoke↔spoke.
- Contrato de entrada del subagente: prompt + payload tipado, nada más.
- Contrato de salida: schema JSON. Si no encaja, error de boundary, no "casi".
- El coordinador es el único que agrega resultados.

Ejemplo mínimo

```
def coordinator(task):
    chunks = split(task)
    results = [run_subagent(prompt=focused_prompt(c), schema=ChunkResult)
               for c in chunks]
    return aggregate(results)            # JSONs tipados, no prosa
```

`run_subagent` arranca sesión nueva, sin pasar `coordinator.history`.

Anti-patrón

"Telepatía compartida": pasar todo el historial del coordinador al subagente "por si acaso". Resultado: contexto diluido, atención degradada, política filtrada a un agente que no tenía por qué verla.

Argumento de certificación

- Sé dibujar el diagrama hub-and-spoke y explicar por qué no hay aristas laterales.
- Sé enumerar qué se pasa al subagente y qué no.
- Sé argumentar por qué la salida tipada es preferible a "resúmen en prosa".

Auto-evaluación

1. ¿Cómo demuestro que el subagente NO recibió historial del coordinador?

`run_subagent` arma `messages=[{"role":"user", ...}]` desde cero por cada llamada. No hay variable `coordinator_history` que se pase. Si quisiera evidencia adicional: imprimir `len(messages)` antes de cada `create` — siempre debe ser 1.

2. ¿Qué hago si dos subagentes producen JSONs en conflicto?

Los registro ambos en el agregado (ver `coordinator_aggregate`: cada `value` viene con `source`). La resolución se delega: si el conflicto importa, escala a humano (Kata 16); si no, queda como dato preservado con provenance (Kata 20). Nunca elijo uno arbitrariamente.

3. ¿Cuándo está justificado romper el aislamiento?

Casi nunca. La excepción razonable es pasar artefactos compartidos read-only (un schema, una guía de estilo) en el system prompt — pero eso es contexto estable, no historial conversacional. Si me veo tentado a pasar history, primero pregunto si el problema se resuelve con un mejor prompt o partiendo distinto la tarea.

Kata 05 — Extracción Estructurada Defensiva con JSON Schema

Concepto

Para extraer datos de texto no estructurado, se fuerza al modelo a usar un **tool call con JSON Schema**. El schema declara: campos obligatorios reales, campos opcionales con uniones nullable, y enums con válvula de escape (`other`, `unclear`) acompañada de un campo `details`.

Por qué importa

Pedirle al modelo "devuélveme JSON" en prosa libre garantiza alucinación silenciosa: inventará campos faltantes, llenará vacíos con `""`, o forzará valores fuera de dominio. El schema es el único contrato que falla cerrado.

Modelo mental

- "Required" significa "siempre presente en la fuente"; si no lo es, márcalo nullable union.
- Default `""` es alucinación: si no sabe, debe ser `null` o `unclear`.
- Enums sin escape obligan al modelo a mentir cuando el valor no encaja.
- `tool_choice` forzado evita que devuelva texto y haga "best-effort prosa".

Ejemplo mínimo

```
{
  "name": "extract_invoice",
  "input_schema": {
    "type": "object",
    "required": ["invoice_id", "currency", "status"],
    "properties": {
      "invoice_id": {"type": "string"},
      "currency": {"type": "string", "enum": ["USD","EUR","COP","other"]},
      "currency_other_details": {"type": ["string","null"]},
      "status": {"type": "string", "enum": ["paid","pending","unclear"]},
      "due_date": {"type": ["string","null"], "format": "date"}
    }
  }
}
```

El cliente valida el resultado y rechaza si no encaja; nunca "ajusta" la salida.

Anti-patrón

JSON pedido en prosa, sin schema, con todos los campos `required` y sin enums con escape. Garantiza fabricación silenciosa: el modelo prefiere inventar a admitir incertidumbre.

Argumento de certificación

- Sé justificar `tool_choice` forzado vs prompt-only.
- Sé explicar la diferencia entre `nullable` y `optional` y por qué importa.
- Sé diseñar enums que admitan ambigüedad sin romper el contrato.

Auto-evaluación

1. Si el dato no aparece en la fuente, ¿qué valor debe poner el modelo?

`null` para campos nullable; el enum de escape (`other`, `unclear`) para enums. **Nunca** un valor inventado plausible.

2. ¿Por qué `category_other_details` es necesario al lado del enum?

Si elige `other`, necesito saber qué quiso decir. Sin el campo paralelo el `other` no aporta información — es básicamente un null. Con él, el agente registra la categoría no contemplada y yo decido si vale la pena agregarla al enum.

3. ¿Qué prueba demuestra que el schema bloquea fabricación cuando reintroduzco `required` excesivos?

Si marco `customer_id` como required, en los casos 2 y 4 el modelo se ve forzado a inventarlo. Comparar la salida del extractor con `customer_id: required` vs nullable demuestra el impacto. La aserción defensiva: pasar un texto sin id explícito y verificar que el extractor con schema correcto devuelve `null`.

Kata 06 — Errores Estructurados en MCP

Concepto

Un servidor MCP que falla NO devuelve un string genérico de error. Devuelve un payload tipado: `isError: true`, `errorCategory`, `isRetryable`, y `explanation`. El agente decide reintento, escalada o aborto a partir de esos campos, no del texto del error.

Por qué importa

`"something went wrong"` obliga al modelo a adivinar qué hacer: reintenta para siempre, abandona, o escala mal. Un error tipado convierte fallos en transiciones deterministas del bucle agéntico (Kata 1).

Modelo mental

- Tres ejes: ¿es error? ¿es reintentable? ¿qué categoría (auth, rate_limit, not_found, validation, transient)?
- El agente lee los flags, no la prosa.
- `explanation` es para el humano que audita el log, no para el modelo.
- Las políticas de retry (backoff, n máximo) se ejecutan en el cliente.

Ejemplo mínimo

```json
{
  "isError": true,
  "errorCategory": "rate_limit",
  "isRetryable": true,
  "retryAfterSeconds": 30,
  "explanation": "API quota exceeded for tenant X"
}
```

Cliente:

```python
if result.isError and result.isRetryable:
    sleep(result.retryAfterSeconds); continue
if result.isError and result.errorCategory == "auth":
    return escalate(result)
```

Anti-patrón

Devolver `raise Exception("failed")` o un string `"Error: ..."` y dejar que el modelo “razone” qué hacer. El agente entrará en bucle, abandonará, o escalará casos que sólo necesitaban backoff.

Argumento de certificación

- Sé enumerar las categorías de error MCP típicas y la respuesta correcta para cada una.
- Sé justificar por qué la decisión de retry vive en el cliente, no en el modelo.
- Sé describir cómo este kata se apoya en Kata 1 (control por señal) y prepara Kata 16 (escalada humana).

Auto-evaluación

1. ¿Cómo trato un error que llega sin `errorCategory`?

Lo trato como `errorCategory="unknown"` y `isRetryable=false` por defecto. Loguear y escalar es preferible a un retry ciego. La defensa es estricta: el cliente sólo confía en lo que el contrato declara.

2. ¿Cuál es la diferencia entre `transient` y `rate_limit` en política de retry?

`transient` es backoff exponencial corto (ms-segundos), `rate_limit` respeta `retryAfterSeconds` que viene del proveedor. Son curvas de espera distintas; la categoría se la doy al ejecutor de retry, no al modelo.

3. ¿Qué prueba reintroduce el anti-patrón (string genérico) y qué assert falla?

Llamar al handler `flaky_string` con el bucle robusto: el bucle no encuentra `isError` / `isRetryable`, asume éxito, y el modelo recibe un `tool_result` falso. Una aserción defensiva: validar el shape del result con un schema antes de pasarlo al historial; falla si llega un string crudo.

Kata 07 — Exploración Segura con Plan Mode

Concepto

Antes de modificar un repositorio desconocido, el agente entra en **Plan Mode**: modo sólo-lectura. Explora, escribe un documento markdown con hallazgos y arquitectura propuesta, y obtiene aprobación humana **antes** de transicionar a ejecución directa (escritura).

Por qué importa

Una refactorización masiva sin reconocimiento previo es destrucción probabilística. Plan Mode separa "entender" de "modificar"; el humano firma el plan, no cada edición.

Modelo mental

- Dos modos: read-only (Plan) y write (Direct). La transición es explícita y registrada.
- En Plan Mode, las herramientas de escritura están deshabilitadas, no sólo desaconsejadas.
- El artefacto de aprobación es texto auditable, no un "ok" verbal.
- Aprobación = firma + plan congelado; cambios al plan re-piden aprobación.

Ejemplo mínimo

```
1. /plan        # entra en read-only
2. agente lee, busca, mapea, redacta plan.md
3. humano revisa plan.md → "approve"
4. agente sale de Plan Mode → ejecuta sólo lo que el plan describe
```

Si el agente intenta escribir en Plan Mode, el hook `PreToolUse` lo deniega.

Anti-patrón

Lanzar el agente con permisos de escritura en un repo desconocido y "ver qué pasa". El primer error borra archivos clave o reescribe convenciones; recuperar es caro.

Argumento de certificación

- Sé describir qué herramientas están permitidas en cada modo.
- Sé justificar por qué la aprobación humana es sobre el plan, no sobre cada edición.
- Sé conectar Plan Mode con Kata 2 (los hooks aplican el modo) y Kata 16 (escalación cuando el plan se invalida).

Auto-evaluación

1. ¿Qué pasa si durante la ejecución el agente descubre que el plan era incorrecto?

Debería detenerse y volver a fase de plan. En esta implementación lo modelaría devolviendo un `tool_result` con `error_code: PLAN_INVALIDATED` y un nuevo turno donde el cliente regresa a Plan Mode (read-only) hasta que se apruebe un plan revisado.

2. ¿Cómo aseguro que Plan Mode bloquea TODA escritura, no sólo las "obvias"?

El cliente pasa explícitamente `tools=READ_ONLY_TOOLS`. No hay forma de que el modelo invoque `apply_patch`: el SDK rechaza tool_use con nombres que no están en la declaración. Adicionalmente, el bucle valida `tu.name in {t["name"] for t in tools}` antes de ejecutar.

3. ¿Dónde se almacena la aprobación humana para que la auditoría la encuentre?

En el log de eventos del bucle, con el plan markdown como adjunto. En producción, en una tabla

`plan_approvals(plan_hash, approver, timestamp)` y el agente en fase 2 verifica que `hash(plan)` coincide con uno aprobado.

Kata 08 — Memoria Jerárquica con `CLAUDE.md`

Concepto

`CLAUDE.md` es la memoria persistente del agente en un proyecto. Vive en tres niveles que se cargan en cascada:

- `~/.claude/CLAUDE.md` — preferencias del usuario.
- `<repo>/CLAUDE.md` — convenciones del equipo.
- `<repo>/<subpath>/CLAUDE.md` — reglas específicas de un módulo.

Y se compone modularmente con `@path/to/file.md`.

Por qué importa

Repetir convenciones en cada prompt cuesta tokens y diverge entre miembros del equipo. `CLAUDE.md` centraliza una sola fuente de verdad por nivel; el agente la carga automáticamente y la usa como contexto estable (caché-friendly, ver Kata 10).

Modelo mental

- Más específico gana. `repo/src/CLAUDE.md` sobreescribe `repo/CLAUDE.md`.
- Lo personal (estilo del usuario) NO va en el repo.
- Lo del equipo NO va en el home del usuario.
- `@imports` mantienen el archivo principal corto y permiten reutilización.

Ejemplo mínimo

```
# Project rules

## Style
@docs/style-guide.md

## Testing
@docs/testing.md

## Forbidden
- never run `pip install` without venv
```

Y en `~/.claude/CLAUDE.md`:

```
# My preferences
- prefer ruff over black
- terse commits
```

Ambos cargan; el del proyecto domina en conflictos sobre el proyecto.

Anti-patrón

- Meter preferencias personales en el `CLAUDE.md` del repo (contamina al equipo).
- Duplicar reglas en system prompt y CLAUDE.md (drift garantizado).
- Un `CLAUDE.md` monolítico de 2000 líneas (degrada caché y atención).

Argumento de certificación

- Sé describir el orden de precedencia entre los tres niveles.
- Sé justificar el uso de `@imports` para modularidad y caché.
- Sé enunciar qué pertenece a cada nivel y qué no.

Auto-evaluación

1. ¿Dónde guardo “este equipo usa pytest, no unittest”?

En el `CLAUDE.md` del proyecto (`<repo>/CLAUDE.md`). Convención del equipo, viaja con el repo.

2. ¿Dónde guardo “yo prefiero respuestas concisas”?

En `~/.claude/CLAUDE.md`. Es preferencia personal y no debe contaminar al equipo.

3. ¿Qué ocurre si el archivo del usuario y el del proyecto se contradicen?

Gana el del proyecto cuando se trata de algo del proyecto (lenguaje, lints, arquitectura). Gana el del usuario cuando es estilo conversacional. La resolución vive en `merge_claude_md`: el orden de los `layers=` en la llamada define la precedencia, y los layers más específicos van después para que su contenido aparezca al final del system prompt (zona de mayor atención, ver Kata 11).

Kata 09 — Reglas Condicionales por Ruta

Concepto

Reglas heurísticas (estilo, lints, convenciones específicas) se cargan **sólo cuando el agente edita archivos que coinciden con un patrón**. No siempre.

Por qué importa

Un `CLAUDE.md` que carga 2000 líneas para todos los archivos paga el costo en todas las sesiones, incluso cuando el agente sólo está editando un README. Cargar reglas Python sólo al tocar `*.py` libera contexto para el resto.

Modelo mental

- La regla declara su `glob` de activación: `src/**/*.py`, `*.tf`, etc.
- El agente carga la regla al entrar al archivo, la descarta al salir.
- Reglas grandes (heurísticas de lenguaje) → condicionales.
- Reglas universales (políticas de seguridad) → siempre cargadas.

Ejemplo mínimo

```
# CLAUDE.md (raíz)
@rules/security.md          # siempre

## When editing src/**/*.py
@rules/python-style.md
@rules/python-testing.md

## When editing infra/**/*.tf
@rules/terraform.md
```

`python-style.md` no se carga al editar `README.md` — no consume tokens.

Anti-patrón

Un único `CLAUDE.md` enorme con todas las heurísticas mezcladas. Cuesta tokens en cada turno, dispersa atención (ver Kata 11), y hace que cambiar una regla afecte sesiones que nada tienen que ver.

Argumento de certificación

- Sé clasificar qué reglas son condicionales y cuáles son universales.
- Sé estimar el ahorro de tokens al condicionar por ruta.
- Sé conectar este kata con Kata 8 (jerarquía) y Kata 10 (caché).

Auto-evaluación

1. ¿La regla "no commits sin tests" es condicional o universal?

Universal — aplica a todo cambio sin importar el lenguaje.

2. ¿Qué pasa si dos reglas condicionales se aplican al mismo archivo?

Ambas se concatenan al system prompt. Si se contradicen, es un bug del autor de las reglas — `assemble_system` debería detectar y advertir, o el equipo debe especializar el glob para no superponerse.

3. ¿Cómo verifico que la regla NO se cargó cuando no debía?

Comparando el system prompt resuelto entre dos targets: el caso B no contiene la sección "Python". Una aserción

defensiva: `assert "type hints" not in assemble_system(["README.md"])`.

Kata 10 — Optimización Económica con Prefix Caching

Concepto

La API de Claude reusa el caché KV cuando el **prefijo** del prompt es idéntico turno a turno. Si organizamos el contexto como **estático primero, dinámico al final**, el primer ~90 % del prompt entra en caché y se factura ~10 % del costo.

Por qué importa

Insertar la fecha actual o el `user_id` al inicio del prompt invalida el caché en cada llamada. Mismo contenido, 10× más caro. El orden importa más que el volumen.

Modelo mental

- Estático = system prompt, CLAUDE.md, definiciones de tools, contexto pesado del repo.
- Dinámico = input del usuario, timestamps, estado efímero.
- Regla: estático arriba, dinámico abajo. El borde dinámico se aísla con tags XML `<reminder>`) para no ensuciar el prefijo.
- Métrica: `cache_creation_input_tokens` vs `cache_read_input_tokens`.

Ejemplo mínimo

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT_BIG_AND_STABLE},   # estático
    {"role": "user", "content": REPO_CONTEXT_BIG},                  # estático
    *prior_turns,                                                   # estático
    {"role": "user", "content": f"<reminder>now: {now}</reminder>\n{user_input}"},
]
```

Mismo system + mismo repo entre llamadas → cache hit.

Anti-patrón

```
content = f"Today is {today}. {SYSTEM_PROMPT}"   #    rompe caché
```

Cualquier valor que cambie por turno al inicio invalida todo lo que sigue.

Argumento de certificación

- Sé enunciar la regla “static-prefix-first, dynamic-suffix-last”.
- Sé interpretar las métricas `cache_creation` vs `cache_read`.
- Sé estimar el ahorro económico esperado (orden de magnitud ≈ 10×).

Auto-evaluación

1. ¿Dónde meto la fecha actual sin romper el caché?

En el sufijo del último mensaje del usuario, idealmente dentro de un tag (`<reminder>now=...</reminder>`). El prefijo (system + history estable) sigue siendo idéntico turno a turno.

2. Si el system prompt cambia un carácter, ¿qué pasa con el caché?

Se invalida — el prefijo deja de coincidir. Por eso los `@imports` (Kata 8) ayudan: cambias un archivo importado y solo se invalida si efectivamente el contenido cambia.

3. ¿Cómo demuestro empíricamente el ahorro?

Comparando `cache_read_input_tokens` entre la versión correcta y la rota. La diferencia es directa: en la celda §4 cache_read=0 en ambos turnos; en §3 cache_read >> 0 en el turno 2.

Kata 11 — Mitigación de Dilución Softmax (Edge Placement + Compactación)

Concepto

La curva de atención del transformer tiene forma de U: lo del inicio y el final del contexto se atiende fuerte; lo del medio se diluye ("lost in the middle"). Por eso las reglas duras se colocan en los **bordes** del prompt, y cuando el contexto pasa ~50 %, se **compacta** antes de que se pierdan.

Por qué importa

Un agente puede estar siguiendo una política perfectamente al turno 5 y violarla al turno 30 sin "olvidarla" — sólo dejó de atenderla porque quedó en el medio. La pérdida es silenciosa y no aparece en logs.

Modelo mental

- Bordes = atención alta. Centro = baja.
- Reglas críticas (seguridad, compliance) → al inicio Y se repiten al final como `<reminder>`.
- Detalles ricos (datos, código) → centro.
- Contexto > 50 % → compactar: resumen estructurado + scratchpad (Kata 18).
- "Compactar" no es "borrar"; es "reescribir en forma densa preservando invariantes".

Ejemplo mínimo

```
[SYSTEM]   reglas críticas duras                    ← borde
[CONTEXT]  archivos, datos, conversación pasada     ← centro
[USER]     pregunta actual
[REMINDER] mismas reglas críticas, reformuladas     ← borde
```

Política de compactación:

```
if usage_fraction(history) > 0.55:
    history = compact(history, preserve=["rules","decisions","escalations"])
```

Anti-patrón

Poner la regla más importante una sola vez, en medio de un system prompt gigante. El día que el contexto crece, la regla queda en el valle de la U y deja de aplicarse — sin error visible.

Argumento de certificación

- Sé describir la curva U y nombrar el efecto "lost in the middle".
- Sé enunciar la regla "bordes para reglas, centro para datos".
- Sé fijar un umbral concreto de compactación (50–60 %) y justificarlo.

Auto-evaluación

1. ¿Por qué repetir la misma regla al inicio y al final no es redundancia?

La curva U significa que ambos extremos tienen alta atención y el centro muy baja. Repetir la regla en los dos bordes garantiza que al menos una copia está en zona de alta atención sin importar dónde crece el contexto.

2. ¿Qué se compacta primero: turnos antiguos del usuario o tool_results intermedios?

Tool_results intermedios cuya conclusión ya quedó incorporada en turnos posteriores (ej. una búsqueda que ya

generó una decisión). Los turnos del usuario antiguos se compactan después, preservando los que contienen reglas o decisiones.

3. ¿Cómo pruebo que una regla “olvidada” se sigue aplicando tras compactar?

Inyectando el ataque (como `ATTACK` arriba) tras la compactación. Si la respuesta sigue rechazando, la regla sobrevivió. Si no, el preservador no la marcó como crítica — bug del filtro `preserve_keywords`.

Kata 12 — Prompt Chaining Multi-Pass

Concepto

Cuando una tarea no cabe cognitivamente (auditar 50 archivos, resumir 200 páginas), se descompone en **pases secuenciales**: pase local por unidad (archivo, página) y luego pase de integración que sólo ve los resúmenes.

Por qué importa

Pedirle al modelo "audita estos 50 archivos" en un solo prompt satura su atención: se pierde detalles, alucina entre archivos, y produce un resumen genérico. Encadenar prompts mantiene cada pase enfocado y barato.

Modelo mental

- Pase 1 (paralelo): por unidad, salida tipada y compacta.
- Pase 2 (integración): sólo ve los outputs del pase 1, no las unidades crudas.
- Cada pase tiene un schema de salida; el siguiente pase consume esos schemas.
- El modelo nunca ve la totalidad cruda.

Ejemplo mínimo

```
# Pase 1 (por archivo)
local = [analyze_file(f, schema=FileFindings) for f in files]

# Pase 2 (integración)
report = integrate(local, schema=AuditReport)
```

`local[i]` ya está condensado: el pase 2 cabe holgadamente.

Anti-patrón

Concatenar todos los archivos en un solo prompt y pedir el reporte final. Resultado: respuesta superficial, alucinaciones cruzadas entre archivos, costo máximo, calidad mínima.

Argumento de certificación

- Sé identificar tareas candidatas para chaining vs single-pass.
- Sé diseñar schemas de transición entre pases.
- Sé conectar este kata con Kata 4 (subagentes para paralelizar el pase 1) y Kata 11 (cada pase respeta el límite de contexto).

Auto-evaluación

1. ¿Cuándo NO conviene chaining (overhead > beneficio)?

Cuando los docs son muy pocos y muy cortos (<2-3 docs cabiendo holgadamente). El overhead de N llamadas no se justifica.

2. ¿Qué pasa si un pase 1 falla en una unidad? ¿se aborta, se omite, se reintenta?

Reintenta una vez (transitorio); si vuelve a fallar, lo registra con `error: true` en el agregado y deja que el pase 2 lo refleje en el reporte. Nunca se omite silenciosamente.

3. ¿Cómo evito que el pase 2 "rellene" lo que el pase 1 dejó vacío?

El system prompt del pase 2 es explícito: "no inventes información que no aparezca en los resúmenes". Y al ser tipados, el pase 2 puede ver `null` o `unclear` y respetarlos.

Kata 13 — Code Review Headless en CI/CD

Concepto

Claude Code corre en CI sin TTY (`claude -p ...`) y produce **JSON estructurado** con anotaciones por línea. El pipeline parsea el JSON con un schema y publica comentarios deterministas en el PR. No hay regex sobre prosa libre.

Por qué importa

Un reviewer humano se cansa, falla, y no escala a 100 PRs/día. Un reviewer LLM en CI encuentra issues consistentes (estilo, secrets, anti-patrones) en cada PR. Pero sólo si su salida es **estructurada** — si no, el pipeline tendrá que parsear prosa y romperá el primer día que el modelo cambie de redacción.

Modelo mental

- `claude -p "prompt" --output-format=json` → JSON contra schema declarado.
- Schema: lista de `Annotation { file, line, severity, rule_id, message }`.
- El runner del CI valida con el schema; si falla, falla el job, no se "ajusta".
- Humano sigue siendo el gate final de merge.

Ejemplo mínimo

```
# .github/workflows/review.yml
- run: |
    claude -p "$REVIEW_PROMPT" \
           --output-format=json \
           --schema annotations.schema.json \
           > out.json
- run: python scripts/post_annotations.py out.json
```

`post_annotations.py` valida con el schema y crea comentarios de PR. Cero parsing de texto.

Anti-patrón

Pedir review en prosa y hacer `grep "ERROR"` sobre el output. Falla el día que el modelo escriba "issue" o "warning" o cambie de idioma. Sin schema, el CI está apostando.

Argumento de certificación

- Sé describir el flag `--output-format=json` y la validación con schema.
- Sé conectar este kata con Kata 5 (extracción defensiva) y Kata 1 (control por señal, no por prosa).
- Sé justificar por qué el humano sigue siendo gate final.

Auto-evaluación

1. ¿Qué hace el CI si el JSON no valida contra el schema?

Falla el job, no "ajusta" la salida. Retorno code != 0 y el revisor humano interviene. Es preferible 1 PR sin review automatizado a 100 PRs con reviews silenciosamente vacíos.

2. ¿Cómo cacheo prompts caros en CI sin invalidar el caché por turno?

System prompt + reglas estables van con `cache_control: ephemeral` (Kata 10). El diff del PR va al final como `user` content. Mismo prefijo entre PRs distintos pero del mismo proyecto = cache_read alto.

3. ¿Qué reviews delego al modelo y cuáles dejo para humano?

Modelo: estilo, secrets, anti-patrones evidentes, lints semánticos. Humano: decisiones arquitectónicas, breaking changes, cambios en contratos públicos. La regla: si requiere conocimiento del negocio, humano.

Kata 14 — Few-Shot para Calibrar Bordes

Concepto

Cuando la tarea es subjetiva (tono, formato no estándar, juicio estético), una descripción zero-shot deja al modelo en su default genérico. **2–4 ejemplos input/output** desplazan su distribución hacia el formato deseado más rápido y más barato que un párrafo de instrucciones.

Por qué importa

Decir "responde en estilo casual chileno" no produce el resultado; mostrar 3 ejemplos de cómo se ve, sí. Few-shot es la forma más eficiente de comunicar "ground truth" para casos sin definición rígida.

Modelo mental

- Los ejemplos son del **mismo schema** que la salida esperada.
- Cubren los **bordes** del dominio, no el caso fácil.
- 2–4 suelen ser suficientes; >5 satura sin mejorar.
- Few-shot complementa (no reemplaza) al schema (Kata 5).

Ejemplo mínimo

```
prompt = """
Clasifica el ticket. Ejemplos:

ticket: "no me llega la factura desde hace 3 meses"
clase: "billing", urgencia: "high"

ticket: "tengo una sugerencia de mejora para la app"
clase: "feedback", urgencia: "low"

ticket: "no puedo entrar, me dice token expirado"
clase: "auth", urgencia: "high"

ahora clasifica:
ticket: "{user_text}"
"""
```

Anti-patrón

- Ejemplos triviales que no representan los bordes (todos casos fáciles).
- Llenar el prompt con 20 ejemplos "por si acaso": dispersa atención (Kata 11), rompe caché (Kata 10), no mejora.
- Mezclar formatos entre ejemplos.

Argumento de certificación

- Sé identificar cuándo few-shot supera a instrucciones en prosa.
- Sé diseñar ejemplos que cubran bordes, no centro.
- Sé combinar few-shot + schema (Kata 5) para tareas subjetivas con formato estricto.

Auto-evaluación

1. ¿Cuándo añadir un ejemplo más empeora el resultado?

Cuando satura la atención (más allá de ~4-5 para muchos casos) o introduce contradicciones con los anteriores. La

curva de mejora es cóncava: los primeros 2-3 valen oro; el quinto rara vez compensa.

2. ¿Por qué los ejemplos van al inicio (estático) y no al final?

Porque son contenido estable: van en el bloque cacheable (Kata 10) y en zona de alta atención (Kata 11). Si los pongo al final, los pierdo del caché y compiten con la pregunta del usuario.

3. Si los ejemplos contradicen el schema, ¿quién gana?

El schema. `tool_choice` con `input_schema` rechaza valores fuera del enum aunque el ejemplo los muestre. Por eso los ejemplos deben ser consistentes con el schema; un ejemplo que use una categoría no listada degrada los resultados.

Kata 15 — Evaluación Crítica y Auto-Corrección

Concepto

Cuando el modelo extrae números (totales, sumas, fechas calculadas), debe **cruzar lo que calculó vs lo que la fuente declara**. Si discrepan, no decide arbitrariamente: emite un flag de conflicto y enruta a revisión humana.

Por qué importa

Un total de factura calculado por el modelo puede coincidir con el declarado… o no. Sin verificación cruzada, el sistema confía silenciosamente en la alucinación más plausible. En facturación o contabilidad, eso es un incidente.

Modelo mental

- Dos fuentes de verdad: lo declarado en el documento y lo calculado por el agente. Deben coincidir.
- Si difieren más allá de un epsilon, marcar `mismatch=true` con ambos valores.
- Nunca "elegir el más razonable". Escalar (Kata 16).
- Aplica a: totales numéricos, sumas, conteos, fechas derivadas.

Ejemplo mínimo

```
{
  "stated_total": 1234.56,
  "computed_total": 1230.00,
  "mismatch": true,
  "delta": 4.56,
  "needs_human_review": true
}
```

Cliente:

```
if abs(stated - computed) > epsilon:
    flag_for_review(invoice, stated, computed)
```

Anti-patrón

Tomar `stated_total` directo o, peor, "corregirlo" silenciosamente al `computed_total` sin avisar. Puede ocultar fraude, errores de OCR, o alucinación del propio modelo.

Argumento de certificación

- Sé identificar campos numéricos sujetos a verificación cruzada.
- Sé definir el epsilon de tolerancia (cero para enteros, ε pequeño para monedas) y justificarlo.
- Sé conectar este kata con Kata 16 (escalada humana) y Kata 20 (provenance).

Auto-evaluación

1. ¿Qué pasa si el documento no declara total (sólo línea por línea)?

`stated_total = null` (campo nullable). `computed_total` se calcula. `mismatch = false` (no hay con qué comparar). `needs_human_review = false`. Estado limpio: el agente reporta sólo lo computado.

2. ¿Cómo distingo "error de OCR" de "fraude" en el flag?

A nivel del flag, no se distinguen — ambos disparan `needs_human_review`. La diferenciación es de proceso: la cola humana tiene heurísticas (delta pequeño y consistente con redondeos vs delta grande sin patrón).

3. ¿Qué prueba reintroduce el anti-patrón (auto-corrección silenciosa) y qué assert falla?

Modificar el system prompt a “si difieren, devuelve `stated_total` y no menciones la diferencia”. Una aserción defensiva en el cliente: si `stated_total` y `computed_total` están ambos presentes y difieren, `mismatch` debe ser `true`. Si el modelo pone `false` con valores discrepantes, falla el assert.

Kata 16 — Protocolo de Handoff a Humano

Concepto

Cuando el agente toca una política que no puede resolver (límite operativo excedido, decisión irreversible, conflicto de datos), invoca la herramienta `escalate_to_human`, **suspende generación de prosa** y emite un payload JSON estricto: `customer_id`, `issue_summary`, `actions_taken`, `escalation_reason`.

Por qué importa

Pasar al humano un transcript crudo de la conversación es desastre operacional: el operador tiene que leer todo, adivinar contexto, y decidir bajo presión. Un payload tipado le da un paquete autocontenido y accionable.

Modelo mental

- Detectar precondición → llamar tool `escalate_to_human`.
- El tool corta la generación de texto y obliga a salida tipada.
- El payload es **autocontenido**: el humano no debería tener que leer nada más.
- Es un end-state del bucle, no una pausa: el agente no continúa hasta que el humano decide.

Ejemplo mínimo

```
{
  "customer_id": "C-1287",
  "issue_summary": "Refund $1500 requested, exceeds tier-2 limit ($1000)",
  "actions_taken": ["validated_identity", "fetched_order_history"],
  "escalation_reason": "policy_limit_refund",
  "recommended_action": "human_approval_for_full_refund"
}
```

Anti-patrón

- Dejar que el modelo “negocie” en prosa con el cliente cuando ya superó la política.
- Pasar al humano `messages[]` crudo y que se las arregle.
- Resumen libre sin schema; el humano interpreta como puede.

Argumento de certificación

- Sé enumerar precondiciones de escalada típicas.
- Sé describir por qué la salida del handoff es tipada y autocontenida.
- Sé conectar este kata con Kata 2 (hook puede forzar `ask_human`) y Kata 15 (mismatch numérico → handoff).

Auto-evaluación

1. ¿Qué hago si el modelo intenta seguir generando prosa después del tool call?

El bucle del cliente, al detectar `tool_use=escalate_to_human`, despacha al humano y termina la sesión. No se inyecta ningún `tool_result` que reanude la conversación: el handoff es un end-state, no una pausa.

2. ¿Cómo aseguro que `actions_taken` refleja realmente lo ejecutado y no alucinación?

El cliente mantiene un log canónico de las acciones que se ejecutaron en el bucle (los `tool_use` aceptados con sus resultados). En el dispatcher, comparo `handoff["actions_taken"]` contra ese log y rechazo el handoff si menciona una acción que nunca corrió.

3. ¿Cuándo es legítimo no escalar y simplemente abortar?

Cuando el costo de resolver excede el valor del caso (peticiones spam, abuso). Aborto = respuesta fija “no procedemos” sin invocar al humano. Debe registrarse para que el operador, si revisa la cola de abortos por muestreo, detecte falsos positivos.

Kata 17 — Procesamiento Masivo con Message Batches API

Concepto

Para cargas que **no son interactivas** (auditorías, backfills, evaluaciones), la Message Batches API procesa miles de requests offline a ~50 % del costo. Cada request lleva un `custom_id` que correlaciona request↔response y aísla fallos.

Por qué importa

Pagar tarifa real-time por trabajo offline es desperdicio. Y procesar 10 000 prompts uno por uno con un `for` rompe rate limits, no maneja fallos parciales, y tarda horas. Batch es el patrón correcto.

Modelo mental

- Batch = colección de requests independientes con `custom_id` único.
- El batch puede acabar `ended` con éxitos parciales: cada request tiene su propio status.
- Se procesa en background, se polea, se descarga el resultado.
- Para fallos masivos, fragmentar el batch (sub-batches) y reintentar selectivamente.

Ejemplo mínimo

```
batch = client.messages.batches.create(requests=[
    {"custom_id": f"audit-{i}", "params": {...}}
    for i, _ in enumerate(items)
])
# poll
while batch.processing_status != "ended":
    sleep(30); batch = client.messages.batches.retrieve(batch.id)
# correlate
for r in client.messages.batches.results(batch.id):
    save(r.custom_id, r.result)
```

Anti-patrón

- Bucle síncrono `for item in 10_000: client.messages.create(...)` : tarifa full, rate limit, sin recuperación de fallos.
- Batch sin `custom_id` o con `custom_id` no único: pierdes correlación.
- Re-procesar todo el batch cuando sólo el 1 % falló.

Argumento de certificación

- Sé identificar cargas elegibles para Batch (offline, latency-tolerant).
- Sé describir el ciclo create → poll → results.
- Sé justificar la importancia del `custom_id` y la fragmentación selectiva.

Auto-evaluación

1. ¿Cuál es el ahorro económico esperado vs API real-time?

~50 % en input/output tokens (consultar pricing actual). Más el ahorro indirecto de no necesitar lógica de retry/backoff propia.

2. ¿Cómo recupero un batch interrumpido sin re-pagar el 99 % exitoso?

Filtrando los `result.type != "succeeded"` y armando un nuevo batch sólo con esos `custom_id`. El éxito previo no se re-procesa.

3. ¿Qué hago si dos requests del batch tienen el mismo `custom_id` por error?

La API lo rechaza al `create`. La defensa: validar localmente con `assert len({r["custom_id"] for r in reqs}) == len(reqs)` antes de enviar.

Kata 18 — Scratchpad Persistente

Concepto

Un archivo (`investigation-scratchpad.md`) externo a la conversación donde el agente vuelca **descubrimientos durables**: hipótesis confirmadas, decisiones, hallazgos de archivos. Sobrevive a `/compact` y a reinicios de sesión.

Por qué importa

Cuando el contexto se compacta (Kata 11), se pierde detalle. Si un descubrimiento crítico vivía sólo en el historial conversacional, desaparece. El scratchpad es memoria persistente curada por el propio agente.

Modelo mental

- Conversación = memoria volátil (puede compactarse o resetearse).
- Scratchpad = memoria persistente (archivo en disco).
- El agente escribe sólo conclusiones validadas, no monólogo.
- Estructurado en secciones: Hipótesis, Decisiones, Hallazgos, Pendientes.
- Al inicio de cada sesión nueva, el agente lee el scratchpad como contexto.

Ejemplo mínimo

```
# Investigation Scratchpad

## Decisiones
- 2026-04-25: usar pydantic v2 (T-19 confirmó compatibilidad).

## Hallazgos
- `src/legacy/parser.py` tiene un bug de offset (línea 142). Replicado.

## Pendientes
- Revisar si `--strict` rompe tests integration-*.
```

Tool de escritura:

```
def append_scratchpad(section: str, entry: str): ...
```

Anti-patrón

- “Mantener todo en la conversación, ya recordará”. Falla al primer `/compact`.
- Escribir el scratchpad en prosa libre y largo: pierde su función de memoria densa.
- Re-leer todo el scratchpad cada turno (rompe caché, ver Kata 10): leer al inicio y referenciar después.

Argumento de certificación

- Sé describir la diferencia entre memoria conversacional y memoria persistente.
- Sé enunciar qué se escribe en el scratchpad y qué no.
- Sé conectar este kata con Kata 11 (compactación) y Kata 19 (investigación adaptativa).

Auto-evaluación

1. ¿Qué pasa si el scratchpad y la conversación se contradicen?

Gana el scratchpad. La conversación es contexto reciente y volátil; el scratchpad es memoria curada. Si la contradicción es genuina, el agente debe actualizar el scratchpad con una nueva entrada que reemplaza la anterior.

2. ¿Cuándo el agente debe borrar** entradas del scratchpad?**

Cuando un hallazgo se invalida (ej. un bug reportado se confirma como falso positivo). En la práctica, agrego una entrada que invalida la anterior antes que borrar — preservar el rastro de la decisión.

3. ¿Cómo verifico que un hallazgo sobrevivió a `/compact`?

Inicio una sesión nueva (sin pasar `messages` previa), incluyo el scratchpad en el system prompt, y le pregunto al modelo por el hallazgo. Si lo recuerda, sobrevivió. La celda §3.1 hace exactamente eso.

Kata 19 — Investigación Adaptativa (Descomposición Dinámica)

Concepto

En un dominio desconocido, no se planifica al detalle de antemano: el agente **mapea topología** primero (búsqueda por nombre y por contenido), genera un plan priorizado, y **re-adapta** cuando un hallazgo invalida el plan vigente. Todo dentro de un presupuesto de exploración acotado.

Por qué importa

Un plan rígido en territorio desconocido garantiza desperdicio: el agente sigue explorando ramas muertas porque "estaba en el plan". Adaptar dinámicamente prioriza atención sobre lo que la realidad muestra, no lo que la hipótesis inicial asumía.

Modelo mental

- Fase 1: mapeo barato (glob de nombres, regex de imports/símbolos).
- Fase 2: priorización (¿qué módulos son centro? ¿cuáles huérfanos?).
- Fase 3: investigación profunda sólo sobre los priorizados.
- Si una hipótesis se rompe, **emitir nuevo plan** y volver a priorizar.
- Presupuesto: número máximo de archivos a leer / queries / minutos.

Ejemplo mínimo

```
topology = scan_repo(globs=["src/**/*.py"])
plan = prioritize(topology)               # heurística declarada
budget = Budget(files=50, queries=20)

while plan and budget.remaining():
    target = plan.pop()
    finding = deep_dive(target, budget)
    if finding.invalidates(plan):
        plan = re_plan(topology, finding)
```

Persistir en scratchpad (Kata 18) cada `finding` y `re_plan`.

Anti-patrón

- Hacer plan completo al minuto 0 y seguirlo aunque la realidad lo contradiga.
- "Read all files" sin presupuesto: se acaba el contexto antes de aprender algo.
- Re-planificar en cada turno por reflejo (no converge nunca).

Argumento de certificación

- Sé definir presupuesto de exploración y justificar el límite.
- Sé enunciar el criterio de re-planificación (qué disparara un re-plan, qué no).
- Sé conectar este kata con Kata 4 (subagentes para deep-dive) y Kata 18 (scratchpad como memoria del proceso).

Auto-evaluación

1. ¿Cómo decido si un hallazgo invalida el plan o sólo lo refina?

Refina si es un detalle dentro del scope actual ("la función está en `auth.py` no en `utils.py`"). Invalida si el scope

cambia ("el problema no es auth, es el middleware"). El umbral es práctico: si requiere mirar archivos no en el plan inicial, es invalidación → re-plan explícito.

2. ¿Qué hago cuando se acaba el presupuesto sin conclusión clara?

Devuelvo lo que tengo + flag `incomplete=true` + lista de pendientes para escalar. Nunca hago una conclusión inventada por presión de presupuesto.

3. ¿Cómo evito loops de re-planificación?

Cada re-plan consume del presupuesto. Si en N pasos no he reducido el espacio de exploración, abort. En código: contar re-plans y limitar a 2-3.

Kata 20 — Preservación de Provenance

Concepto

Cada afirmación factual extraída de fuentes mantiene un **mapeo tipado a su origen**: `claim`, `source_id`, `source_name`, `publication_date`. Los conflictos entre fuentes NO se resuelven en silencio: se marcan y se enrutan a humano.

Por qué importa

Tras agregar contenido de muchas fuentes vía subagentes (Kata 4), perder el hilo de "quién dijo qué" hace imposible auditar el resultado. Resúmenes en prosa libre **se ven correctos** y aluciinan sin que se note. Provenance es la única defensa.

Modelo mental

- No hay claim sin source. Es un invariante de schema.
- Si dos fuentes contradicen, se registran ambas bajo `conflict: true`, no se promedia ni se elige.
- La fecha de publicación importa: fuente más reciente no siempre gana, pero el humano necesita verla.
- El reporte final preserva la lista de claims tipados; la prosa, si existe, se genera**a partir** de esa lista, no la sustituye.

Ejemplo mínimo

```
{
  "claim": "ARR Q3 2025 = 12M USD",
  "sources": [
    {"id": "doc-A", "name": "Annual Report 2025", "date": "2025-12-01"},
    {"id": "doc-B", "name": "Investor Deck", "date": "2025-09-15"}
  ],
  "conflict": false
}
```

Conflicto:

```
{
  "claim": "Headcount end-2025",
  "sources": [
    {"id":"doc-A","name":"Report","date":"2025-12-01","value":"450"},
    {"id":"doc-C","name":"HR export","date":"2026-01-10","value":"462"}
  ],
  "conflict": true,
  "needs_human_review": true
}
```

Anti-patrón

- Resumen agregado en prosa, sin citas, sin source_id. Imposible auditar.
- "Resolver" el conflicto eligiendo la fuente que parece más oficial.
- Asumir que la fuente más nueva siempre gana.

Argumento de certificación

- Sé enunciar el invariante "no hay claim sin source".
- Sé describir la política de conflictos (registrar ambos, escalar).
- Sé conectar este kata con Kata 4 (agregación tras subagentes), Kata 15 (verificación numérica) y Kata 16

(escalada humana).

Auto-evaluación

1. ¿Qué hago si la fuente no tiene fecha?

`source_date` se marca null o `unknown`. El claim sigue siendo válido pero el humano que resuelve un conflicto verá la ausencia de fecha como debilidad de la fuente.

2. ¿Cuándo dos sources con el mismo `value` cuentan como "una sola confirmación" y cuándo como dos?

Si comparten origen primario (mismo dataset, mismo reporte oficial copiado en dos lugares), una. Si son extracciones independientes con cadenas de evidencia distintas, dos. La heurística práctica: registrarlos por separado y dejar que el humano lo lea.

3. ¿Qué prueba reintroduce el anti-patrón (prosa sin citas) y qué assert falla?

Reemplazar la salida del extractor por prosa libre. Una aserción defensiva: para cada claim final, `assert "source_id" in claim and claim["source_id"] in known_doc_ids`. Si el modelo emite un claim sin source_id (o con uno fabricado), el assert falla y el pipeline rechaza la respuesta.

Glosario

Agentic Loop. `while` controlado por `stop_reason`; `tool_use` ejecuta y continúa, `end_turn` corta, otros valores cortan con motivo explícito. Núcleo del Kata 01.

Anti-patrón. Implementación común que parece funcionar y falla en cuanto cambia el input. Cada kata tiene uno; documentarlo es parte del entregable.

Bootstrap (`katas._shared.bootstrap`). Función que pide la API key con `getpass`, devuelve cliente envuelto con guardia de presupuesto. Primera celda de todo notebook del workshop.

Budget guard. Contador que envuelve `client.messages.create` y aborta con `BudgetExceeded` antes de quemar la cuenta.

Cache control (`cache_control: ephemeral`). Marcador en bloques de contenido que indica al SDK marcar el segmento como cacheable. Se usa para implementar prefix caching (Kata 10).

Cascada de `CLAUDE.md`. Orden de precedencia: usuario → proyecto → módulo. Más específico gana. Composable con `@imports` (Kata 08).

Custom ID (Batch API). Identificador único por request en un batch que permite correlación 1:1 entre input y output (Kata 17).

Determinismo. Misma entrada → misma transición. Sólo señales estructuradas (`stop_reason`, hook verdict) lo garantizan; texto generado no (Principio I).

Edge placement. Estrategia de ubicar reglas críticas al inicio Y al final del contexto para sobrevivir la curva U de atención (Kata 11).

Escape enum. Valor del enum (`other`, `unclear`) acompañado de un campo `_details` que permite al modelo admitir ambigüedad sin romper el contrato (Kata 05).

Evento estructurado. Diccionario tipado `(iter, stop_reason, branch, …)` emitido por iteración. Permite reconstruir la traza sin re-llamar al modelo.

Few-shot. 2-4 ejemplos input/output en el prompt para calibrar la distribución del modelo en tareas subjetivas (Kata 14).

Forced `tool_choice`. Parámetro `tool_choice={"type":"tool","name":"X"}` que obliga al modelo a invocar el tool X. Garantiza salida tipada por schema (Kata 05, 13).

Hub-and-spoke. Topología de subagentes: un coordinador central, N subagentes aislados, cero aristas laterales. Cero historial compartido (Kata 04).

Lost in the middle. Efecto medible donde la atención del transformer decae en el centro del contexto. Mitigación: edge placement (Kata 11).

Mismatch flag. Campo booleano que el extractor pone en `true` cuando valores declarado y computado difieren más de un epsilon. Dispara revisión humana (Kata 15).

MCP (Model Context Protocol). Estándar para herramientas/servidores que expone errores tipados (`isError`, `errorCategory`, `isRetryable`). El cliente decide retry o escalada desde flags, no desde prosa (Kata 06).

Plan Mode. Modo read-only previo a ejecución directa. El cliente entrega distinto set de tools en cada fase (Kata 07).

Prefix caching. Reutilización del KV cache cuando el prefijo del prompt es idéntico turno a turno. Regla práctica: estático arriba, dinámico abajo (Kata 10).

PreToolUse / PostToolUse hook. Función que corre antes (gate) o después (normalización) de la ejecución de la herramienta. Aplica políticas en código, no en prompt (Katas 02, 03).

Provenance. Mapeo `claim → source` preservado tras agregación. Conflictos no se resuelven en silencio (Kata 20).

Scratchpad. Archivo markdown estructurado donde el agente vuelca descubrimientos durables. Sobrevive a `/compact` (Kata 18).

`stop_reason`. Metadato estructurado del Message que indica por qué terminó el turno. Único árbitro legítimo del control de flujo (Kata 01).

Subagente. Llamada nueva e independiente al SDK con prompt mínimo y salida tipada. Cero memoria heredada del coordinador (Kata 04).

Cheat sheet (una página)

#	Kata	Regla central
01	Bucle agéntico	Control por `stop_reason`, jamás por texto del modelo
02	PreToolUse	Política en código, prompt sólo sugiere
03	PostToolUse	El modelo nunca ve datos crudos heterogéneos
04	Subagent isolation	Hub-and-spoke; subagente recibe prompt mínimo, devuelve JSON tipado
05	Defensive extraction	Schema con `tool_choice` forzado, enum con escape, nullable union
06	MCP errors	`isError` + `errorCategory` + `isRetryable` tipados, retry en cliente
07	Plan Mode	Tools de lectura → plan firmado → tools de escritura
08	CLAUDE.md memory	Cascada user → project → module; `@imports` para modularidad
09	Path rules	Reglas heurísticas se cargan sólo cuando el target hace match
10	Prefix caching	Estático arriba (cacheable), dinámico abajo
11	Softmax mitigation	Reglas duras en bordes; compactar al 50–60 % de capacidad
12	Prompt chaining	Pase 1 tipado por unidad → pase 2 integra sin ver crudos
13	Headless CI review	`claude -p` con schema; `jsonschema.validate` antes de publicar
14	Few-shot	2-4 ejemplos de bordes, no del centro fácil
15	Self-correction	`mismatch` y `needs_human_review` son required, no se autocorrige
16	Human handoff	Tool con schema estricto y enum de motivo; suspende generación de prosa
17	Batch processing	`custom_id` único, polling/webhook, recuperación selectiva
18	Scratchpad	Memoria persistente fuera de la conversación; sobrevive a `/compact`
19	Adaptive investigation	Topology → prioritize → deep-dive con presupuesto y re-plan
20	Data provenance	Cada claim con `source_id`; conflictos preservados, no resueltos

Antes del examen, repite

1. *El control de flujo es por señal estructurada, nunca por prosa.*
2. *Los schemas fallan cerrados; los prompts son sugerencias.*
3. *Estático arriba, dinámico abajo; bordes para reglas, centro para datos.*
4. *Subagentes con prompt mínimo y salida tipada; cero historial heredado.*
5. *Cuando dudes, escala al humano con payload tipado.*